

Contents

guide_GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers **1**

- 0. 先给你一个抓手 1
- 1. 当时的语境: 为什么 GPTQ 重要 2
- 2. 读论文前必须补齐的概念 2
- 3. 论文地图 3
- 4. 从张量形状开始读 4
- 5. naive rounding 为什么会失败 4
- 6. OBG: GPTQ 的直接前身 5
- 7. GPTQ 的三步改造 5
 - 7.1 Step 1: 任意固定顺序 5
 - 7.2 Step 2: lazy batch update 6
 - 7.3 Step 3: Cholesky reformulation 6
- 8. Algorithm 1 用中文重写 6
- 9. 校准和部署流程 7
- 10. 实验证据链 8
 - 10.1 小模型: GPTQ 是否只是快但不准 8
 - 10.2 量化时间: 175B 是否真的可做 8
 - 10.3 OPT WikiText2: 3-bit RTN 崩溃, GPTQ 稳住 9
 - 10.4 BLOOM WikiText2: 同样趋势, 但更容易量化 9
 - 10.5 175B 汇总: 多任务是否稳 9
 - 10.6 实际速度: 为什么低 bit 会更快 10
 - 10.7 group size 和 2-bit 极限 10
- 11. 数学直觉再压缩一次 10
- 12. 和本仓库代码的连接 11
- 13. 这篇论文没有证明什么 11
- 14. 和后续方法的关系 12
- 15. 用 AI agent 学这篇论文的正确方式 12
- 16. 读完后的闭卷复述模板 13

guide_GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers

原论文: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers
作者: Elias Frantar, Saleh Ashkboos, Torsten Hoefer, Dan Alistarh
会议版本: ICLR 2023
本地原文 PDF: learning/quantization-deploy/paper/01_gptq.pdf
本地导读 PDF: learning/quantization-deploy/paper/guide_01_gptq.pdf
本地代码: learning/quantization-deploy/src/gptq_minimal.py
本地机制实验: learning/quantization-deploy/src/gptq_original_minimal.py

0. 先给你一个抓手

GPTQ 要解决的问题可以用一句话说清楚:
大语言模型的权重太大, FP16 放不进少数 GPU; 直接把每个权重四舍五入到 3-bit 或 4-bit 会把输出误

差逐层放大；GPTQ 用校准数据估计每一层输入方向的重要性，然后按列量化权重，并把当前列造成的误差补偿到后续未量化的列上。

如果你只记住一个机制，记这个：

full precision layer:

```
output = W x
```

naive rounding:

```
Q = round(W)
```

```
output_error = W x - Q x
```

GPTQ:

```
quantize one column
```

```
measure that column's error
```

```
use  $H^{-1}$  to move the error into later columns
```

```
repeat until the matrix is quantized
```

这里的 H 来自校准激活 x 。它不是训练时的真实 loss Hessian，而是 layer reconstruction objective 的二阶近似。新手最容易误读 GPTQ 的地方，是以为它只是一个更聪明的 rounding。更准确的说法是：它把 rounding 变成了一个有输入分布意识的二阶误差分配问题。

1. 当时的语境：为什么 GPTQ 重要

GPTQ 出现时，开源社区正处在一个很具体的工程拐点上。OPT-175B、BLOOM-176B 这类公开大模型已经出现，但 FP16 权重的显存需求非常夸张。论文开头用 GPT-3 175B 做参照：175B 参数用 FP16 存储约 326GB。只看这个数字就能理解问题：模型不是“慢一点”这么简单，而是常规单卡根本放不下。

部署大模型至少有三类显存压力：

- 权重显存：模型参数本身占用最大，175B 级别会超过单卡容量很多。
- KV cache 显存：自回归生成时，每层都要缓存过去 token 的 key/value。
- 临时激活和运行时开销：即使推理 batch size 为 1，也需要 buffer 和 kernel 工作空间。

当时已有的方向包括：

- FP16 多卡推理：精度保留，但硬件门槛高，通信和调度复杂。
- 8-bit 权重量化：例如 LLM.int8()，显存下降明显，但 175B 仍然可能需要多张大卡。
- 训练感知量化或蒸馏：可能效果好，但要训练或微调，非常昂贵。
- 普通 post-training quantization：不训练，只用少量校准数据，但大模型低 bit 下容易崩。

GPTQ 的野心就在这里：不重新训练，不用任务标签，只用少量校准文本，把 175B 级别模型压到 3-bit 或 4-bit，并且让困惑度和下游任务尽量接近 FP16。

这件事对今天仍然重要。你现在看到的很多 4-bit 本地模型、量化部署、显存换吞吐的系统工程，都离不开这条问题线：权重是瓶颈，低 bit 是出路，但 naive rounding 不够。

2. 读论文前必须补齐的概念

Post-training quantization, 简称 PTQ:

模型已经训练完，不再做完整训练。量化阶段只拿少量校准数据，通过校准激活来决定 scale、zero point 或权重调整。GPTQ 属于 PTQ。

RTN, round-to-nearest:

把权重映射到量化网格上最接近的值。它便宜、稳定、容易并行，但完全不看“这个权重误差会不会伤害输出”。论文把 RTN 当成主要 baseline，因为它是当时可扩展到超大 LLM 的强工程基线。

Layer-wise reconstruction:

不直接优化整模型最终 loss，而是逐层处理。对某一层，给定输入激活 x 和原权重 w ，希望找到量化权重 $\hat{w}$ ，让这一层的输出尽量不变。

target:

$\text{minimize } ||W X - \hat{W} X||_2^2$

Hessian proxy:

在这个 layer reconstruction 目标下，二阶信息可以从 $x x^T$ 或按实现形状写作 $x^T x$ 得到。直觉是：如果校准数据经常激活某个输入方向，那个方向上的权重误差就更重要。

OBQ:

Optimal Brain Quantization。它继承 Optimal Brain Surgeon 的思想，用二阶信息决定量化某个权重后，应该怎样更新剩余权重来补偿误差。OBQ 精度好，但逐权重量化太慢，不能直接处理 175B 大模型。

Group size:

把连续权重分组，每组有自己的量化 scale 和 zero point。group 越小，网格越灵活，精度更好，但额外 scale/zero 存储更多，kernel 也更复杂。

Weight-only quantization:

GPTQ 主要压缩权重。激活仍然可以用 FP16，推理时动态反量化权重再做矩阵乘。这和同时量化 activation 的方案不同。

3. 论文地图

这篇论文的结构很清楚，可以按下面顺序读：

- Section 1: 说明为什么超大 LLM 的 3/4-bit PTQ 是有价值的问题。
- Section 2: 放到量化文献中，区分训练量化、PTQ、OBQ、LLM.int8() 等方向。
- Section 3: 给出 layer-wise reconstruction 目标，并回顾 OBQ 的二阶补偿公式。
- Section 4: GPTQ 算法本体。三步改造：任意顺序、lazy batch update、Cholesky reformulation。
- Algorithm 1: 完整伪代码。读懂它，基本就读懂了 GPTQ。
- Section 5: 实验证据。先小模型对 PTQ，再大模型量化时间，再 OPT/BLOOM 的 PPL 和零样本任务。
- Section 6: 总结与限制。重点是速度来自减少内存搬运，不是减少数学乘法数量。
- Appendix: 更多 PTB/C4/LAMBADA/PIQA/ARC/StoryCloze 数字，以及实现细节。

图表也可以这样定位：

- Figure 1: GPTQ、RTN、FP16 在 OPT/BLOOM 上的直观对比，展示 3-bit RTN 崩溃和 GPTQ 稳定性。
- Figure 2: GPTQ 的按列和按块量化图示，是理解 Algorithm 1 的关键。
- Table 1: 小模型上和 AdaRound、BRECQ、OBQ 等 PTQ 方法比较。
- Table 2: 大模型完整量化时间，证明 GPTQ 能扩展到 175B。
- Table 3 和 Table 4: OPT/BLOOM 在 WikiText2 上的核心 PPL 结果。
- Table 5: OPT-175B 和 BLOOM-176B 的多任务汇总。
- Table 6: 3-bit OPT-175B 实际生成延迟和 GPU 数量减少。
- Figure 3: LAMBADA 零样本准确率。
- Figure 4 和 Table 7: group size 与极低 bit 量化。

4. 从张量形状开始读

先不要急着背公式。把一层线性层画成张量:

```
W: [out_features, in_features]
X: [in_features, num_tokens]      # 论文写法之一
Y: [out_features, num_tokens]
```

```
full precision:
    Y = W X
```

```
quantized:
    Y_hat = W_hat X
```

```
reconstruction error:
    ||Y - Y_hat||_2^2
```

在本仓库的 PyTorch toy code 里, 为了习惯 batch-first 写法, 校准激活常写成:

```
X: [num_samples, in_features]
H = X^T X / num_samples
output = W @ X^T
```

形状换了, 意思不变。关键是 H 的维度必须是:

```
H: [in_features, in_features]
```

为什么是 $\text{in_features} \times \text{in_features}$? 因为 GPTQ 按输入维度的列来量化 W 。量化第 j 列时, 它要知道第 j 个输入方向和后续输入方向之间的相关性。如果校准激活里两个输入方向高度相关, 那么当前列的误差可以通过调整另一个列的一部分来抵消。

你可以把 Figure 2 翻译成下面这个文本图:

weight matrix W

```
columns:
    c0 c1 c2 c3 c4 c5 c6 c7 ...
```

```
block i:
    [c0 c1 c2 c3] are processed inside the block
    c0 is quantized first
    its error updates c1 c2 c3 immediately
    after the block is done, one batched update touches c4 c5 ...
```

```
H^-1 information:
    tells how error from column cj should be pushed into later columns
```

这就是 GPTQ 的张量级直觉: 不是压一个标量, 而是在一个矩阵里沿列推进, 把误差沿 Hessian inverse 的方向分配出去。

5. naive rounding 为什么会失败

RTN 的做法是:

```
for every weight w:
    q = nearest_grid_value(w)
```

它默认每个权重的误差彼此独立。但 Transformer 线性层不是这样工作的。一个输出通道是很多输入方向的加权和:

$$y = w_0 x_0 + w_1 x_1 + \dots + w_k x_k$$

如果 x_0 和 x_1 在真实数据中经常一起变化, 那么 w_0 的量化误差可能通过微调 w_1 的未量化值来补偿。RTN 完全不利用这个机会。

在 8-bit 时, 单个权重误差小, RTN 常常还能工作。在 4-bit, 误差变大; 在 3-bit, 网格更粗, 某些模型上 RTN 会让 PPL 爆炸。论文的 OPT/BLOOM 表格正是在证明: 低 bit 大模型不是简单 round 一下就完事。

6. OBQ: GPTQ 的直接前身

OBQ 的目标仍然是 layer-wise reconstruction:

```
minimize ||W X - W_hat X||_2^2
```

它按每一行 w 独立处理。对某一行来说, OBQ 每次选一个权重量化, 并更新剩余还没量化的权重来补偿误差。

论文里给出的核心思想可以改写成更易读的形式:

```
choose the next weight q:
    prefer the weight with small normalized rounding damage
```

```
rounding damage:
    (quant(w_q) - w_q)^2 / Hinv[q, q]
```

```
compensate the remaining weights:
    delta_F = - (w_q - quant(w_q)) / Hinv[q, q] * Hinv[F, q]
```

注意这里的 $Hinv[q, q]$ 。它不是随便除一个数, 而是在问: 当前这个维度对重构误差有多敏感。分母越大, 当前权重的误差越容易被吸收; 分母越小, 直接 round 它的代价就更大。

OBQ 很自然, 但慢。对一个 $drow \times dcol$ 的矩阵, 原始 OBQ 的输入维度相关复杂度很高。论文指出, 把它直接拿来量化十亿到千亿参数模型, 工程上不可接受。

所以 GPTQ 的问题不是 “怎样发明量化”, 而是:

```
Can we keep the second-order compensation idea,
but make it fast enough for 175B-scale Transformers?
```

7. GPTQ 的三步改造

7.1 Step 1: 任意固定顺序

OBQ 是 greedy 的: 每一步都找当前最适合量化的权重。这个选择看起来合理, 但它让不同输出行的量化顺序不一样, 于是 Hessian inverse 的更新也要按行单独做, 成本很高。

GPTQ 的第一个观察很关键:

在大而过参数化的层里, 用任意固定顺序量化, 最终误差和 greedy 顺序差距通常不大。

这个观察改变了算法结构。既然可以固定顺序, 那么所有输出行都可以按同一个列顺序量化:

```

for column j:
    quantize W[:, j] for all output rows together
    compute one shared  $H^{-1}$  column effect
    update W[:, j+1:]

```

这一步把“逐权重”变成“逐列”。因为 H 只依赖输入激活 x ，不依赖输出行的具体权重，所有行可以共享同一个 Hessian inverse 信息。论文说这带来几个数量级的可扩展性改进。

7.2 Step 2: lazy batch update

直接按列更新仍然会遇到 GPU 利用率问题。每量化一列，就更新巨大矩阵的后续部分，计算量不一定大，但内存读写很多，容易被 memory bandwidth 卡住。

GPTQ 的第二步是 lazy batch update。直觉是：

量化当前列时，只需要当前列已经收到正确补偿；更后面的列不必立刻全局更新。

于是算法一次处理一个 block，例如论文常用 $B = 128$ 列：

```

inside a block:
    process columns one by one
    update only the rest of this block immediately
    store each column's error in E

```

```

after the block:
    apply one batched update to all later columns

```

这不改变理论计算量太多，但极大改善 GPU 上的吞吐，因为它把许多小而分散的更新合并成更适合矩阵运算的批更新。

7.3 Step 3: Cholesky reformulation

大模型上反复更新 inverse Hessian 会有数值问题。论文指出，矩阵可能因为累计误差变得 indefinite，导致补偿方向错掉，某些层量化结果非常坏。

小模型上，加 dampening 可能就够了：

```
H_damped = H + lambda * mean(diag(H)) * I
```

但大模型需要更稳健的做法。GPTQ 注意到，量化第 q 个权重时真正需要的是 H_{inv} 当前状态的第 q 行从对角线往后的部分。这个信息可以通过 Cholesky 形式预先稳定地得到，而不是每一步显式做高风险的 inverse 删除更新。

所以 Algorithm 1 开头会把 inverse Hessian 信息转换成 Cholesky 形式。对读者来说，不必把所有线性代数细节一次吃完。你先抓住设计目的：

- 避免反复显式更新 inverse matrix。
- 保留量化当前列所需的 $H_{inv}[j, j:]$ 信息。
- 用成熟 Cholesky kernel 提升数值稳定性和速度。

8. Algorithm 1 用中文重写

论文 Algorithm 1 的输入：

W: weight matrix to quantize
Hinv: inverse of damped Hessian proxy
B: block size

输出:

Q: quantized weight matrix

核心流程:

```
Q = zeros_like(W)
E = zeros([out_features, B])
Hinfo = Cholesky(Hinv)^T

for each block starting at column i:
    for j in columns inside this block:
        Q[:, j] = quant(W[:, j])

        E[:, j - i] =
            (W[:, j] - Q[:, j]) / Hinfo[j, j]

        W[:, j : i+B] =
            W[:, j : i+B] - outer(E[:, j-i], Hinfo[j, j : i+B])

    W[:, i+B :] =
        W[:, i+B :] - E @ Hinfo[i : i+B, i+B :]
```

这个伪代码里最重要的是三种矩阵:

W:
working full precision matrix
later columns are continually adjusted

Q:
committed quantized columns
once a column enters Q, it is fixed

E:
error buffer inside the current block
used for one lazy global update after the block

张量级别看, $E @ Hinfo[...]$ 是把一个 block 里所有列产生的误差一次性推到后续列。它就是 lazy batch update 的落地。

9. 校准和部署流程

论文的 calibration setup 很克制:

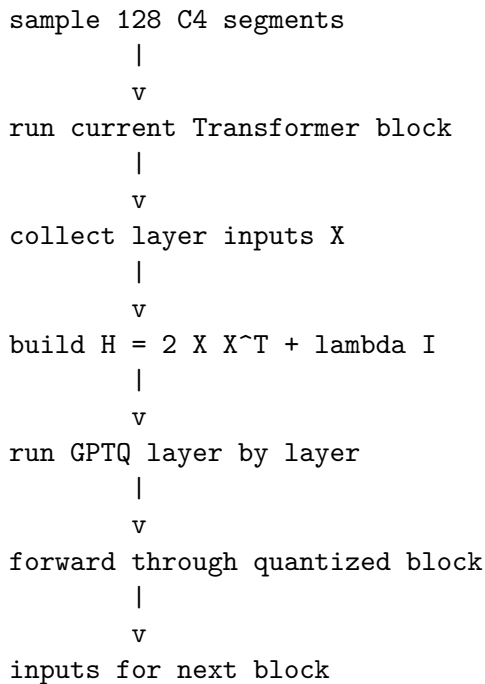
- 数据: 从 C4 随机抽 128 个 2048-token 片段。
- 标签: 不需要任务标签。
- 目标: 只用 generic text 捕捉模型各层输入分布。
- 硬件: 量化 175B 级别模型时使用单张 80GB A100。
- 量化网格: 标准 uniform per-row asymmetric min-max grid。

量化整模型时，作者不是把完整 FP16 175B 模型一次性塞进 GPU，而是按 Transformer block 流式处理。论文写到每次加载一个 block，通常包括 6 层，累计这些层的 Hessian，再量化。

还有一个容易漏掉的实现细节：

量化完当前 block 后，校准数据会通过已经量化的 block 再 forward 一次，产生下一个 block 的输入。这意味着后续层看到的是“前面已经有量化误差”的真实输入，而不是全 FP16 模型的理想输入。这个细节提升了结果，而且成本很小。

整体流程可以画成：



10. 实验证据链

10.1 小模型: GPTQ 是否只是快但不准

Table 1 在 ResNet18 和 ResNet50 上对比 AdaRound、AdaQuant、BRECQ、OBQ、GPTQ。它的目的不是证明 GPTQ 专为视觉模型，而是回答一个方法学问题：

如果和已有高精度 PTQ 方法比，GPTQ 会不会为了速度牺牲太多精度？

结论：

- 4-bit 下，GPTQ 和 AdaRound、BRECQ、OBQ 大体同一档。
- 3-bit 下，GPTQ 比最好的方法略弱，但仍明显强于 AdaQuant。
- 更重要的是，GPTQ 用少于 1 分钟的量化时间，换来接近慢方法的精度。

这给后面扩展到 LLM 打基础：GPTQ 不是只有速度没有精度。

10.2 量化时间: 175B 是否真的可做

Table 2 是工程证据。单张 A100 上：

- OPT-13B: 约 20.9 分钟。
- OPT-30B: 约 44.9 分钟。
- OPT-66B: 约 1.6 小时。

- OPT-175B: 约 4.2 小时。
- BLOOM-1.7B: 约 2.9 分钟。
- BLOOM-3B: 约 5.2 分钟。
- BLOOM-7.1B: 约 10.0 分钟。
- BLOOM-176B: 约 3.8 小时。

这张表是 GPTQ 的核心贡献之一。很多量化方法可以在小模型上漂亮，但量化 175B 如果要几周，就不是同一个问题。GPTQ 证明了二阶 PTQ 可以进入超大模型尺度。

10.3 OPT WikiText2: 3-bit RTN 崩溃，GPTQ 稳住

Table 3 给出 OPT 家族在 WikiText2 上的 PPL。

关键数字:

- OPT-175B FP16: 8.34。
- OPT-175B RTN 4-bit: 10.54。
- OPT-175B GPTQ 4-bit: 8.37。
- OPT-175B RTN 3-bit: 约 7.3e3，基本崩溃。
- OPT-175B GPTQ 3-bit: 8.68。

这组数字很好地说明了 GPTQ 的故事。4-bit 下，GPTQ 几乎贴着 FP16；3-bit 下，RTN 变成不可用，而 GPTQ 仍然给出合理 PPL。

还有一个细节值得记：论文观察到更大的模型通常更容易量化，但 OPT-66B 是例外。作者指出它早期层有较多 dead units，可能导致压缩更难。这提醒我们：量化不是只由参数规模决定，模型内部激活结构也重要。

10.4 BLOOM WikiText2: 同样趋势，但更容易量化

Table 4 是 BLOOM 家族。

关键数字:

- BLOOM-176B FP16: 8.11。
- BLOOM-176B RTN 4-bit: 8.37。
- BLOOM-176B GPTQ 4-bit: 8.21。
- BLOOM-176B RTN 3-bit: 571。
- BLOOM-176B GPTQ 3-bit: 8.64。

BLOOM 的 RTN 4-bit 比 OPT 看起来没那么差，说明模型家族差异会影响量化难度。但 3-bit RTN 仍然崩，GPTQ 仍然能保持可用。

10.5 175B 汇总: 多任务是否稳

Table 5 聚焦 OPT-175B 和 BLOOM-176B，覆盖 WikiText2、PTB、C4 和 LAMBADA。

OPT-175B:

- FP16 在 WikiText2 上是 8.34。
- GPTQ 4-bit 是 8.37。
- GPTQ 3-bit 是 8.68。
- GPTQ 3-bit 加 group size 128 后，WikiText2 约 8.45。

BLOOM-176B:

- FP16 在 WikiText2 上是 8.11。
- GPTQ 4-bit 是 8.21。
- GPTQ 3-bit 是 8.64。
- GPTQ 3-bit 加 group size 128 后, WikiText2 约 8.26。

这张表还包含 LAMBADA。一个有趣现象是, GPTQ 4-bit 在某些 LAMBADA 数字上甚至不低于 FP16。不要把这理解成量化提升了模型本质能力, 更合理的解释是评测有噪声, 量化扰动有时会偶然改变离散预测。结论应该保守读: GPTQ 在这些任务上没有显著破坏模型能力。

10.6 实际速度: 为什么低 bit 会更快

Table 6 讲的是生成延迟。论文关注 OPT-175B 的 batch size 1 自回归生成, 这时主要瓶颈是 matrix-vector product。

关键数字:

- A100 80GB: FP16 约 230ms/token, 3-bit 约 71ms/token, 约 3.24 倍加速。
- A6000 48GB: FP16 约 589ms/token, 3-bit 约 130ms/token, 约 4.53 倍加速。

这里一定要读清楚: GPTQ 的速度收益主要来自减少 memory movement。它没有把矩阵乘的数学本质变成更少的乘加, 而是用更少 bit 存权重, 推理时动态反量化, 减少从显存搬运的数据量。

这也是为什么它适合自回归生成中的 matrix-vector。batch size 1 时算子常常受内存带宽限制, 权重更小就能更快喂给计算单元。

10.7 group size 和 2-bit 极限

Figure 4 和 Table 7 说明 GPTQ 可以和更细粒度的 group quantization 结合。group 越小, 每组 scale/zero 更贴合局部分布, 低 bit 精度更好。

论文在极端 2-bit 附近也做了探索。结论不是 “2-bit 已经稳了”, 而是:

- 对 175B 大模型, 小 group 可以把 2-bit 附近结果推到看起来有希望的范围。
- group size 带来额外存储开销, 所以平均 bit 不再是纯 2.0。
- ternary 形式可能对特殊硬件有价值。

新手读这里要保持克制。3/4-bit 是主结果, 2-bit 更像方向性证据。

11. 数学直觉再压缩一次

把一行权重的量化看成二次目标:

$$\text{loss}(w_hat) = ||w \cdot X - w_hat \cdot X||_2^2$$

因为这是二次型, 局部误差可以用 Hessian 描述。对某个权重 w_j , round 之后产生误差:

$$\text{raw_error} = w_j - \text{quant}(w_j)$$

GPTQ 不让这个误差原地堆积, 而是用 inverse Hessian 做补偿:

$$\begin{aligned} \text{err} &= (w_j - q_j) / \text{Hinv}[j, j] \\ W_remaining &= W_remaining - \text{outer}(\text{err}, \text{Hinv}[j, remaining]) \end{aligned}$$

这个式子可以这样理解:

- $w_j - q_j$: 当前列因为量化损失了多少。
- $\text{Hinv}[j, j]$: 当前维度自身的二阶敏感度归一化。

- `Hinv[j, remaining]`: 当前维度和后续维度的耦合关系。
- `outer(...)`: 对所有输出行同时做补偿。

如果你能把这四个点讲清楚，GPTQ 的核心数学已经过关。

12. 和本仓库代码的连接

本专题代码有两个层次。

第一层是主课程最小实现:

`learning/quantization-deploy/src/gptq_minimal.py`

它展示:

- `calibrate_hessian(X)`: 用校准激活构造 $X^T X / N$ 。
- `gptq_quantize(W, H)`: 按列量化, 计算误差, 用 `H_inv` 更新后续列。
- 测试文件 `src/tests/test_quant.py`: 对比 GPTQ toy result 和 naive quantization。

第二层是我为这篇导读补的 paper-shaped 机制实验:

`learning/quantization-deploy/src/gptq_original_minimal.py`

`learning/quantization-deploy/src/tests/test_gptq_original_minimal.py`

它更明确地对应论文概念:

- `make_symmetric_row_grid`: 每个输出行一个 scale, 接近论文 per-row grid 的精神。
- `calibration_hessian`: 加入 damped Hessian proxy。
- `gptq_columnwise`: 用列循环和 Hessian inverse 误差补偿复现算法骨架。
- `reconstruction_mse`: 直接评估 $W X$ 和 $Q X$ 的输出差异。
- `toy_comparison`: 固定随机种子, 观察 naive rounding 和 GPTQ compensation 的差别。

运行方式:

```
.\\.venv\Scripts\python.exe `
learning\quantization-deploy\src\gptq_original_minimal.py

.\\.venv\Scripts\python.exe `
learning\quantization-deploy\src\tests\test_gptq_original_minimal.py

.\\.venv\Scripts\python.exe `
learning\quantization-deploy\src\tests\test_quant.py
```

读代码时不要期待它复现论文的 175B 数字。它的作用是把机制降到你能亲手 inspect 的尺度:

paper claim:

low-bit rounding damage should be compensated by second-order information

toy code question:

after quantizing column j , where is its error stored,
and how is it pushed into $W[:, j+1:]$?

这个问题答清楚, 比盯着一个 toy PPL 数字更重要。

13. 这篇论文没有证明什么

GPTQ 很强, 但它不是万能压缩定理。

它没有证明:

- 所有模型在 3-bit 都稳定。
- 任意校准数据都足够好。
- activation quantization 已经解决。
- speedup 与所有硬件、所有 batch size、所有 kernel 都无关。
- bias、安全性、长尾行为在量化后保持不变。
- 2-bit 可以普遍可靠部署。

论文自己也强调, 速度提升主要来自减少内存搬运, 不是减少数学计算本身。对于 matrix-matrix、大 batch 或不同硬件, 收益可能变化。它还主要研究 generative tasks 和标准 accuracy/perplexity, 没系统研究压缩对 bias 或其他二级指标的影响。

工程上还有几个现实限制:

- 需要校准数据和逐层 forward。
- 量化时间虽然可接受, 但不是零成本。
- 低 bit 推理需要合适 kernel, 否则显存省了, 速度未必上去。
- group size、scale 存储、packing layout 会影响最终部署复杂度。
- 很多后续方法会在 GPTQ 的基础上补 activation outlier、weight saliency 或 kernel 细节。

14. 和后续方法的关系

GPTQ 之后, 你会看到很多量化路线:

- AWQ: 更强调 activation-aware 的权重重要性, 保护少量 salient weights。
- SmoothQuant: 把 activation outlier 的困难迁移到权重上, 为 W8A8 服务。
- LLM.int8(): 8-bit 权重量化和 outlier 处理的重要工程基线。
- bitsandbytes NF4: 常用于 QLoRA, 服务训练和微调场景。
- FP8: 更偏训练和硬件生态, 尤其是 Hopper 之后。
- KV cache quantization: 压的是自回归生成中的 cache, 不是主权重。

GPTQ 在这条线里的位置是: 它把“无需训练的大模型低 bit 权重量化”推到了一个可信工程尺度, 并让二阶补偿重新变成 LLM 部署里的实用工具。

15. 用 AI agent 学这篇论文的正确方式

不要让 agent 直接“总结 GPTQ”。你要让它反复考你, 而且每次只考一个点。

可以这样提问:

我正在学习 GPTQ。

请不要泛泛总结。

你一次只问我一个问题。

问题必须围绕下面六类之一:

1. layer reconstruction objective
2. H 和 H^{-1} 的直觉
3. OBQ 到 GPTQ 的三步改造
4. Algorithm 1 的张量形状
5. Table 3/4/5/6 的证据链
6. 本仓库 gptq_original_minimal.py 的代码对应关系

我回答后, 你要指出我的漏洞,

并要求我把答案映射到论文的某个 section 或本地某个函数。
最后让我用 200 字闭卷复述这篇论文。

你还可以让 agent 做“反向讲解”：

请假装我是新手。

你不要先讲 GPTQ。

先给我一个 naive rounding 会失败的二维 toy case。

然后一步步引出为什么需要 H^{-1} 。

每一步都让我先预测输出误差会怎样变化。

真正掌握的标志不是你能背出 GPTQ 的缩写，而是你能闭卷回答：

- 为什么 RTN 在 3-bit 下会崩。
- 为什么 H 来自校准激活而不是来自标签。
- 为什么 GPTQ 可以按列处理所有输出行。
- lazy batch update 为什么主要解决 GPU memory throughput。
- Cholesky reformulation 在防什么数值问题。
- Table 3、Table 4 和 Table 6 分别支撑论文哪三个 claim。
- 本仓库 toy code 里的 `err` 和 `W_work[:, j+1:]` 对应论文 Algorithm 1 的哪一行。

16. 读完后的闭卷复述模板

你可以按这个模板复述：

GPTQ 的问题背景是：

...

它不像 RTN 那样：

...

它先把每层量化写成：

...

OBQ 给它的启发是：

...

为了扩展到 175B，GPTQ 做了三步：

1. ...
2. ...
3. ...

最关键的实验结果是：

...

它的限制是：

...

我能在本仓库看到的最小实现是：

...

如果你能在不看 guide 的情况下填完这段，并且能打开 `gptq_original_minimal.py` 指出每个函数对应论文哪一段，这篇 GPTQ 就算真正进脑子了。